LeanCode

# State Management

How to wrap it up in my app?

# Ephemeral vs global

- Local
- Has my button been touched?
- Short-living in terms of a feature
- Selected tab in bottom navigation
- Current page in the IndexedStack

- Global
- Am I signed in?
- Long-living in terms of a feature, business process or the whole app
- User Authentication state
- Cart items state

We build digital products.

LeanCode

# We already know about ephemeral state

# Ephemeral state

```dart
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'You have pushed the button this many times:',
            ),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headline4,
            ),
```

```dart
class _DraggableCardState extends State<DraggableCard>
    with SingleTickerProviderStateMixin {
  AnimationController? _controller;
  Alignment _dragAlignment = Alignment.center;
  Animation<Alignment>? _animation;

  void _runAnimation(Offset pixelsPerSecond, Size size) {
    _animation = _controller!.drive(
      AlignmentTween(
        begin: _dragAlignment,
        end: Alignment.center,
      ),
    );

    final unitsPerSecondX = pixelsPerSecond.dx / size.width;
    final unitsPerSecondY = pixelsPerSecond.dy / size.height;
    final unitsPerSecond = Offset(unitsPerSecondX, unitsPerSecondY);
    final unitVelocity = unitsPerSecond.distance;

    const spring = SpringDescription(
      mass: 30,
      stiffness: 1,
      damping: 1,
    );

    final simulation = SpringSimulation(spring, 0, 1, -unitVelocity);

    _controller!.animateWith(simulation);
  }

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this);

    _controller!.addListener(() {
      setState(() {
        _dragAlignment = _animation!.value;
      });
    });
  }
```

We build digital products.

LeanCode

# Things that are tightly coupled with our UI

# What about our current pizza order in the app?

# Is it connected to UI?

# Do we want to keep it in some widget?

# What about text field?

# What about text field in three page form?

# ChangeNotifier

```dart
class Counter with ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
    notifyListeners();
  }
}
```

```dart
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (context) => Counter()),
  ],
  child: const MyApp(),
),
```

```dart
floatingActionButton: FloatingActionButton(
  onPressed: () => context.read<Counter>().increment(),
  child: const Icon(Icons.add),
),
```

```dart
class Count extends StatelessWidget {
  const Count({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Text(
      '${context.watch<Counter>().count}',
      key: const Key('counterState'),
      style: Theme.of(context).textTheme.headline4,
    );
  }
}
```

# Pretty simple, huh?

# It’s just a class so we can do whatever we like

# What if…?

```
class Counter with ChangeNotifier {
  int _count = 0;

  int get count => _count;

  void increment() {
    _count++;
  }
}
```

We build digital products.

LeanCode

# Doesn’t standardize

# 10 devs 100 solutions

We build digital products.

LeanCode

```dart
class ValueNotifier<T> extends ChangeNotifier implements ValueListenable<T> {
  /// Creates a [ChangeNotifier] that wraps this value.
  ValueNotifier(this._value);

  /// The current value stored in this notifier.
  ///
  /// When the value is replaced with something that is not equal to the old
  /// value as evaluated by the equality operator ==, this class notifies its
  /// listeners.
  @override
  T get value => _value;
  T _value;
  set value(T newValue) {
    if (_value == newValue)
      return;
    _value = newValue;
    notifyListeners();
  }

  @override
  String toString() => '${describeIdentity(this)}($value)';
}
```

We build digital products.

LeanCode

# Flutter's answer: MVVM

- ViewModel = ChangeNotifier
- Commands for user actions
- provider for dependency injection
- Immutable models (freezed)

https://docs.flutter.dev/app-architecture

We build digital products.

LeanCode

# Many ways

- Provider + ChangeNotifier
- Riverpod
- Bloc / Cubit
- signals, MobX, GetX

# Riverpod

```dart
final counterProvider = NotifierProvider<Counter, int>(Counter.new);

class Counter extends Notifier<int> {
  @override
  int build() => 0;

  void increment() => state++;
}
```

```dart
class CounterText extends ConsumerWidget {
  const CounterText({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('$count');
  }
}
```

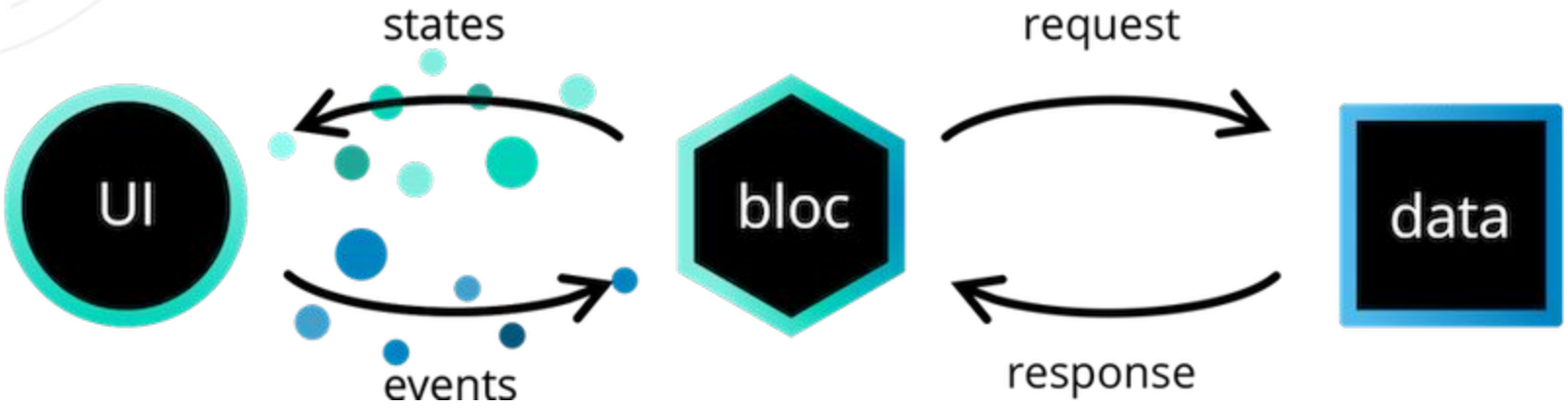
```dart
onPressed: () => ref.read(counterProvider.notifier).increment(),
```

```dart
runApp(const ProviderScope(child: MyApp()));
```

We build digital products.

LeanCode

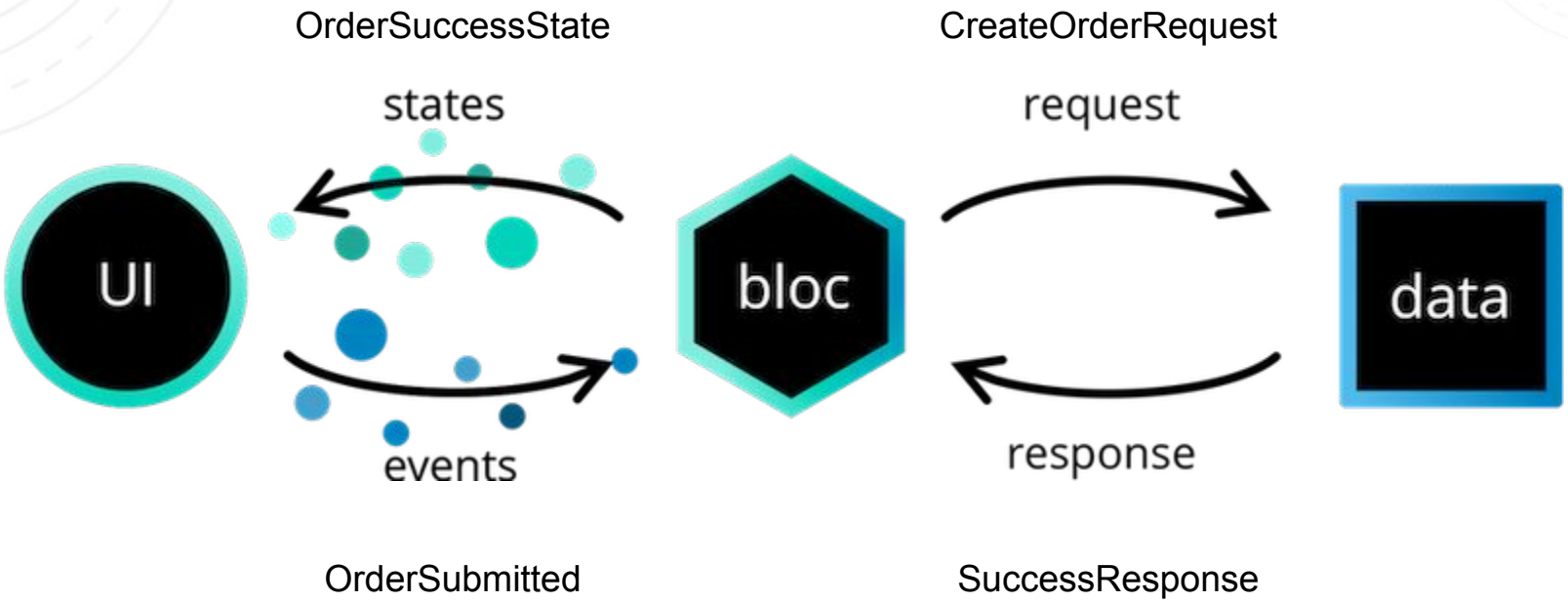
# Bloc




We build digital products.

LeanCode

# Bloc




We build digital products.

LeanCode

# Bloc

```dart
abstract class CounterEvent {}
class Increment extends CounterEvent {}
class Decrement extends CounterEvent {}
```

```dart
class CounterBloc extends Bloc<CounterEvent, int> {
  CounterBloc() : super(0) {
    on<Increment>((event, emit) => emit(state + 1));
    on<Decrement>((event, emit) => emit(state - 1));
  }
}
```

We build digital products.

LeanCode

# Bloc

```
Padding(
  padding: const EdgeInsets.symmetric(vertical: 5.0),
  child: FloatingActionButton(
    child: const Icon(Icons.add),
    onPressed: () =>
        context.read<CounterBloc>().add(Increment()),
  ),
),
```

```
return MaterialApp(
  theme: theme,
  home: BlocProvider(
    create: (_) => CounterBloc(),
    child: CounterPage(),
  ),
);
```

```
child: BlocBuilder<CounterBloc, int>(
  builder: (context, count) {
    return Text(
      '$count',
      style: Theme.of(context).textTheme.headline1,
    );
  },
),
```

We build digital products.

LeanCode

# Event transformers

```dart
class SearchBloc extends Bloc<SearchEvent, List<String>> {
  SearchBloc() : super([]) {
    // new query cancels the previous search
    on<QueryChanged>(_onQueryChanged, transformer: restartable());
    // taps while submitting are ignored
    on<SubmitPressed>(_onSubmit, transformer: droppable());
  }
}
```

```dart
// package:bloc_concurrency
concurrent() // default
sequential()
droppable()
restartable()
```

We build digital products.

LeanCode

# UI = f(state)

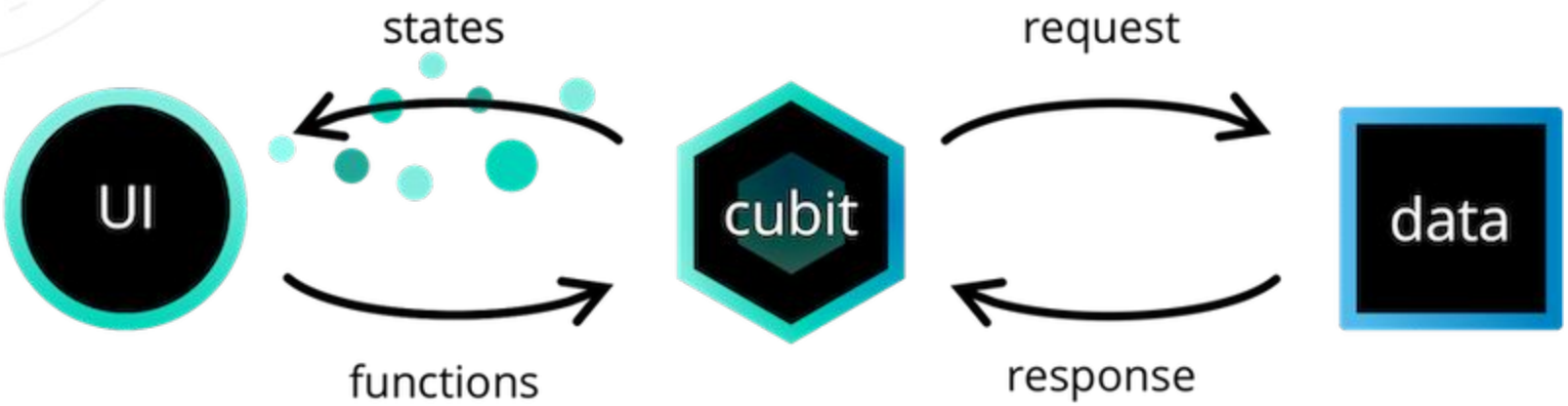
We build digital products.

LeanCode

# UI = f(state)

```
return BlocBuilder<
  BaseRequestCubit<dynamic, TOut, TError>,
  RequestState<TOut, TError>
>(
  bloc: cubit,
  builder: (context, state) {
    return switch (state) {
      RequestInitialState() => const LoadingWidget(),
      RequestLoadingState() => const LoadingWidget(),
      RequestErrorState() => Column(
        children: [
          Text('Something has gone wrong'),
          TextButton(child: Text('retry'), onPressed: onErrorCallback),
        ],
      ),
      RequestEmptyState() => const Text('Nothing here'),
      RequestRefreshingState(:final data) => builder(context, data),
      RequestSuccessState(:final data) => builder(context, data),
    };
  },
);
```

# Cubit




We build digital products.

LeanCode

# Cubit

```
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
}
```

```dart
sealed class RankingListState {
  const RankingListState();

  const factory RankingListState.initial() = RankingListStateInitial;

  const factory RankingListState.inProgress() = RankingListStateInProgress;

  const factory RankingListState.ready({required RankingLineDTO? me}) =
      RankingListStateReady;

  const factory RankingListState.failure() = RankingListStateFailure;
}
```

```dart
class RankingListStateInitial extends RankingListState {
  const RankingListStateInitial();
}

class RankingListStateInProgress extends RankingListState {
  const RankingListStateInProgress();
}

class RankingListStateReady extends RankingListState {
  const RankingListStateReady({required this.me});

  final RankingLineDTO? me;
}

class RankingListStateFailure extends RankingListState {
  const RankingListStateFailure();
}
```

We build digital products.

LeanCode

```dart
class RankingListCubit extends Cubit<RankingListState> {
  RankingListCubit({required RankingsCqrs rankingsCqrs})
      : _rankingsCqrs = rankingsCqrs,
        super(const RankingListState.initial());

  final RankingsCqrs _rankingsCqrs;

  static final _logger = Logger('RankingListCubit');

  Future<void> fetch() async {
    if (state is RankingListStateInProgress) {
      return;
    }

    emit(const RankingListState.inProgress());

    try {
      final rankingResult = await _rankingsCqrs.get(ContestRanking());
      final ranking = rankingResult.dataOrNull;

      if (ranking == null) {
        _logger.severe('Returned ranking was null');
        emit(const RankingListState.failure());
        return;
      }

      emit(RankingListState.ready(me: ranking.me));
    } catch (e, st) {
      _logger.info('Failed to fetch ranking', e, st);
      emit(const RankingListState.failure());
    }
  }
}
```

# Immutable state

```
class CartState {
  CartState(this.items);

  final List<String> items;
}

class CartCubit extends Cubit<CartState> {
  CartCubit() : super(CartState([]));

  void add(String item) {
    state.items.add(item);
    emit(state); // same object: ignored from the 2nd emit on
  }
}
```

```
class CartState extends Equatable {
  const CartState(this.items);

  final List<String> items;

  @override
  List<Object?> get props => [items];
}

class CartCubit extends Cubit<CartState> {
  CartCubit() : super(const CartState([]));

  void add(String item) =>
      emit(CartState([...state.items, item]));
}
```

We build digital products.

LeanCode

# “Nail factory” :)

# leancode_cubit_utils

We build digital products.

LeanCode

```
RequestLayoutConfigProvider(
  onLoading: (context) => const LoadingWidget(),
  onError: (context, error, retry) => Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('Error: $error'),
        const SizedBox(height: 16),
        ElevatedButton(onPressed: retry, child: const Text('Retry')),
      ],
    ),
  ),
```

```
body: RequestCubitBuilder<List<({String url})>, int>(
  cubit: context.watch<DogListRequestCubit>(),
  onInitial: (context) => EmptyDogListWidget(
    onFetch: () => context.read<DogListRequestCubit>().run(),
  ),
  onErrorCallback: () => context.read<DogListRequestCubit>().run(),
  onSuccess: (context, dogs) => DogListDataWidget(
    dogList: FetchedDogList(dogs: dogs),
    onFetch: () => context.read<DogListRequestCubit>().refresh(),
  ),
),
```

We build digital products.

LeanCode

```dart
import 'package:leancode_cubit_utils_cqrs/leancode_cubit_utils_cqrs.dart';

class RankingListCubit extends QueryCubit<RankingPageDTO?, RankingLineDTO?> {
  RankingListCubit({required RankingsCqrs rankingsCqrs})
      : _rankingsCqrs = rankingsCqrs,
        super('RankingListCubit');

  final RankingsCqrs _rankingsCqrs;

  @override
  RankingLineDTO? map(RankingPageDTO? data) => data?.me;

  @override
  Future<QueryResult<RankingPageDTO?>> request() =>
      _rankingsCqrs.get(ContestRanking());
}
```

We build digital products.

LeanCode

# Sources

https://flutter.dev
https://dartpad.dev
https://bloclibrary.dev
https://docs.flutter.dev/app-architecture
https://riverpod.dev
https://pub.dev/packages/leancode_cubit_utils
https://pub.dev/packages/bloc_concurrency